Tutorial III.V - Modelling the Transportation Problem with JuMP

Optimization with Julia



Introduction

Welcome to this tutorial on the transportation problem using JuMP! As always, don't worry if you're new to optimization - we'll walk through everything step by step using a real-world example.

Imagine you're running a solar panel distribution company. You have several warehouses (suppliers) and need to ship solar panels to various solar farms (customers). Each delivered truckload earns revenue, but it also causes variable costs and transportation costs. Your goal is to plan the shipments that maximize your total profit.

By the end of this tutorial, you'll be able to:

1. Understand what a transportation problem is
2. Set up a transportation problem using JuMP
3. Solve the problem and interpret the results

Let's start by loading the necessary packages:

```
using JuMP, HiGHS
using DataFrames, CSV
```

Section 1 - Understanding the Transportation Problem

The transportation problem involves:

- Suppliers (our warehouses)
- Customers (solar farms)
- Transportation costs between each supplier and customer
- Supply available at each warehouse
- Demand required by each solar farm

Our goal is to decide how many truckloads of solar panels to ship from each warehouse to each solar farm to maximize the total profit. Note that the demand of a solar farm is an upper limit, not an obligation: if delivering to a farm would lose money, we are allowed to deliver less than requested - or nothing at all.

Let's set up our problem:

- The revenue for each truckload of solar panels is 11000

- The variable costs for each truckload of solar panels are 6300
- Each delivered truckload thus earns a margin of 11000 - 6300 = 4700, before we pay for the transport
- The available panels at the supplier are given in the file `available-panels.csv`
- The requested panels at the customer are given in the file `panel-demand.csv`
- The transportation costs between suppliers and customers are given in the file `cost.csv`

> **Note**
> We can use the `CSV.read` function to load the data from a CSV file into a DataFrame. If we want to access the directory of the current file, we can again use the convenient `@__DIR__` macro.

> **Tip**
> Ensure you download the three datasets (Panels, Demand, Costs) that are linked on this tutorial page and save them in a folder named `data` within the same directory as your current script. To download a dataset, right-click the corresponding link and choose `Download linked file`. No preprocessing is required, as this tutorial focuses solely on the modeling process.

```
# Fixed parameters
revenue = 11000  # Revenue per truckload of solar panels
varCosts = 6300  # Variable costs per truckload

# Load data from CSV files
available = CSV.read("$(@__DIR__)/data/available-panels.csv", DataFrame)
requested = CSV.read("$(@__DIR__)/data/panel-demand.csv", DataFrame)
travelCosts = CSV.read("$(@__DIR__)/data/cost.csv", DataFrame)

println("Data loaded successfully!")
```

```
Data loaded successfully!
```

Now, we can check out the data by printing the first few rows of each DataFrame. We can use the `first` function to get the first few rows of a DataFrame.

```
println("Available panels:")
first(available,5)
```

```
Available panels:
```

```
5×2 DataFrame
 Row │ supplier   truckloads_available
     │ DataStri…  Int64
─────┼──────────────────────────────────
   1 │ a_1                         478
```

```
   2 | a_2                          371
   3 | a_3                          361
   4 | a_4                          415
   5 | a_5                          354
```

```
println("Requested panels:")
first(requested,5)
```

```
Requested panels:
```

```
5×2 DataFrame
 Row | solar_farm  truckload_demand
     | DataString  Int64
-----|-----------------------------
   1 | b_1                       9
   2 | b_2                      31
   3 | b_3                      47
   4 | b_4                      46
   5 | b_5                      12
```

```
println("Travel costs:")
first(travelCosts,5)
```

```
Travel costs:
```

```
5×3 DataFrame
 Row | supplier   solar_farm  costs
     | DataStri…  DataStri…   Int64
-----|-------------------------------
   1 | a_1        b_1          8052
   2 | a_2        b_1         11845
   3 | a_3        b_1          6103
   4 | a_4        b_1         11099
   5 | a_5        b_1          5625
```

Exercise 1.1 - Understand the Data

Take a moment to look at the data. Can you answer these questions?

1. How many warehouses do we have? Save the number in a variable called `num_warehouses`.
2. How many solar farms are we supplying? Save the number in a variable called `num_solar_farms`.

```
# YOUR ANSWERS BELOW
# Hint: Use the `nrow()` function to count rows


# Test your understanding
@assert num_warehouses == nrow(available)
@assert num_solar_farms == nrow(requested)

println("Great job! Here are the answers:")
println("Number of warehouses: ", num_warehouses)
println("Number of solar farms: ", num_solar_farms)
```

Section 2 - Using dictionaries to store the data

Now, DataFrames are not a very convenient format for our purposes. We have several options now on how to deal with these data sets. As our suppliers and customers are given names, it might be useful to convert the data into dictionaries. Dictionaries are a great data structure that allow us to store key-value pairs, where the keys are unique identifiers and the values are the data associated with those keys. By using dictionaries, we can easily access and modify the data associated with a specific key.

```
available_dict = Dict(
    available.supplier .=> available.truckloads_available
)
requested_dict = Dict(
    requested.solar_farm .=> requested.truckload_demand
)
travelCosts_dict = Dict(
    (row.supplier,row.solar_farm) => row.costs
    for row in eachrow(travelCosts)
)
```

> **Tip**
> You can use the `Dict` function to create a dictionary from two arrays or DataFrames. For example: `Dict(keys .=> values)` will create a dictionary where the keys are the elements of the `keys` array and the values are the elements of the `values` array.

Now, let us check out the dictionaries. We can use the `first` function to get the first few key-value pairs of a dictionary.

```
println("Available panels:")
first(available_dict,5)
```

```
Available panels:
```

```
5-element Vector{Pair{DataStrings.DataString, Int64}}:
  "a_8" => 160
 "a_16" => 111
 "a_40" => 137
 "a_10" => 232
 "a_33" => 226
```

```
println("Requested panels:")
first(requested_dict,5)
```

```
Requested panels:
```

```
5-element Vector{Pair{DataStrings.DataString, Int64}}:
 "b_308" => 6
 "b_397" => 29
 "b_706" => 48
 "b_900" => 48
 "b_226" => 49
```

```
println("Travel costs:")
first(travelCosts_dict,5)
```

```
Travel costs:
```

```
5-element Vector{Pair{Tuple{DataStrings.DataString, DataStrings.DataString},
Int64}}:
 ("a_94", "b_975") => 3377
 ("a_78", "b_996") => 9753
 ("a_98", "b_461") => 10003
 ("a_28", "b_721") => 5286
  ("a_7", "b_754") => 1470
```

Remember, we can also access the value associated with a specific key in a dictionary by using the key inside square brackets. For example: `available_dict["a_1"]` will return the value associated with the key `"a_1"`.

```
print("Value associated with supplier 'a_1': ")
available_dict["a_1"]
```

```
Value associated with supplier 'a_1':
```

```
478
```

Our travel costs dictionary is a bit more complex, as it is a dictionary with tuples as keys. We can access the value associated with a specific supplier and customer by using two

keys inside square brackets. For example: `travelCosts_dict[("a_1","b_1")]` will return the value associated with the keys `("a_1","b_1")`.

```
print("Value associated with supplier 'a_1' and customer 'b_1': ")
travelCosts_dict[("a_1","b_1")]
```

```
Value associated with supplier 'a_1' and customer 'b_1':
```

```
8052
```

We can also extract the keys and values of a dictionary using the `keys` and `values` functions, as shown in the previous tutorial.

```
println("Keys of the travel costs dictionary:")
first(keys(travelCosts_dict),5)
```

```
Keys of the travel costs dictionary:
```

```
5-element Vector{Tuple{DataStrings.DataString, DataStrings.DataString}}:
 ("a_94", "b_975")
 ("a_78", "b_996")
 ("a_98", "b_461")
 ("a_28", "b_721")
 ("a_7", "b_754")
```

```
println("Values of the travel costs dictionary:")
first(values(travelCosts_dict),5)
```

```
Values of the travel costs dictionary:
```

```
5-element Vector{Int64}:
  3377
  9753
 10003
  5286
  1470
```

Dictionaries make it a lot easier to access the data later on, as we can use the keys to directly access the desired value in our model. This will be useful when we want to define the constraints later on.

Section 3 - The model instance

After the preprocessing and data loading, we now can create the model instance with the HiGHS optimizer.

Exercise 3.1 - Creating the model instance

From the last tutorial, you should know how to do this. Create a model instance called `transport_model` and set the optimizer to HiGHS.

```
# YOUR CODE BELOW


# Test your answer
@assert typeof(transport_model) == JuMP.Model
println("Model instance created successfully!")
```

Section 4 - Defining the model

Before we write any code, let's look at the model we want to build. It is a profit maximization model:

$$
\begin{aligned}
\text{Maximize} \quad & F = \sum_{i \in \mathcal{I}} \sum_{j \in \mathcal{J}} (r - c - c_{i,j}) \times X_{i,j} \\
\text{subject to:} \quad & \sum_{j \in \mathcal{J}} X_{i,j} \leq a_i && \forall i \in \mathcal{I} \\
& \sum_{i \in \mathcal{I}} X_{i,j} \leq b_j && \forall j \in \mathcal{J} \\
& X_{i,j} \geq 0 && \forall i \in \mathcal{I}, j \in \mathcal{J}
\end{aligned}
$$

Here, $\mathcal{I}$ is the set of warehouses indexed by i and $\mathcal{J}$ is the set of solar farms indexed by j. The parameters are the revenue r per truckload, the variable costs c per truckload, the transportation costs $c_{i,j}$ per truckload from warehouse i to solar farm j, the available truckloads a_i at warehouse i, and the requested truckloads b_j at solar farm j. The variable $X_{i,j}$ is the number of truckloads shipped from warehouse i to solar farm j.

> **Note**
> Could this model become infeasible? No! Both constraints are $\leq$ constraints, so shipping nothing at all ($X_{i,j} = 0$ everywhere) always satisfies them - even though total supply (30,321 truckloads) and total demand (28,014 truckloads) do not match. And as no warehouse can ship more than it has available, the profit cannot grow without bound either. The solver will therefore always find an optimal solution for this model.

Define the variables

We can now define the variables of our model. We need to define a variable for each warehouse and solar farm pair. As before, we can use the `@variable` macro to define the variables. The syntax is `@variable(model, varname[index1,index2] >= 0)`, where `model` is the model instance, `varname` is the name of the variable, and `index1` and `index2` are the indices of the variable. We can use vectors as input for the indices, but we could also use the keys of the dictionaries. In the following code block we mixed both options, to show you that it is possible.

```
# Define variable
@variable(
    transport_model,
    X[available.supplier,keys(requested_dict)] >= 0
)
```

> ⚠ **Warning**
> Mixing indexing styles like this only works because `available.supplier` and `keys(available_dict)` contain exactly the same names - just as `requested.solar_farm` and `keys(requested_dict)` do. If one of them were a filtered DataFrame or an outdated dictionary, we would get a `KeyError` - or worse, a silently smaller model without any error message. In your own models, it is safer to pick one style and stick to it.

Exercise 4.1 - Define the objective

Now it is your turn to define the objective! We want to maximize the total profit: each truckload shipped from warehouse `i` to solar farm `j` earns the margin of 4700 (revenue minus variable costs) and costs the transportation costs of that route. As before, we can use the `@objective` macro. The syntax is `@objective(model, Max, expression)`, where `model` is the model instance, `Max` indicates that we want to maximize the expression, and `expression` is the expression we want to maximize.

```
# YOUR CODE BELOW
# Hint: Sum over all pairs, e.g. with
# for i in keys(available_dict), j in keys(requested_dict)
# and look up the transportation costs with travelCosts_dict[(i,j)]


# Test your answer
@assert objective_sense(transport_model) == MOI.MAX_SENSE "The objective should
be maximized, not minimized."
@assert isapprox(
    coefficient(objective_function(transport_model), X["a_1","b_1"]),
    revenue - varCosts - travelCosts_dict[("a_1","b_1")];
    atol = 1e-6
) "The profit for each truckload from a_1 to b_1 should be the revenue minus
```

```
the variable costs minus the transportation costs of that route."
println("Objective defined successfully!")
```

Exercise 4.2 - Restrict the available truckloads

Next, we define the constraints with the `@constraint` macro. The first constraint ensures that each warehouse ships at most the number of truckloads it has available. Create it and name it `restrictAvailable`. The syntax is `@constraint(model, name[index], expression)`, where `model` is the model instance, `name[index]` creates one constraint for each element of the index set, and `expression` is the expression we want to constrain. To practice the use of dictionaries, use the keys of the dictionaries for the indices here.

```
# YOUR CODE BELOW
# Hint: For each warehouse i, the sum of X[i,j] over all solar farms j
# has to be lower than or equal to available_dict[i]
```

```
# Test your answer
@assert all(
    is_valid(transport_model, restrictAvailable[i]) for i in
keys(available_dict)
) "The model should contain one constraint restrictAvailable for each
warehouse."
@assert normalized_rhs(restrictAvailable["a_1"]) == available_dict["a_1"] "The
right-hand side of the constraint for warehouse a_1 should be its available
truckloads, $(available_dict["a_1"])."
println("Supply constraint defined successfully!")
```

Exercise 4.3 - Restrict the requested truckloads

The second constraint ensures that each solar farm receives at most the number of truckloads it requested. Create it and name it `restrictDemand`. Note the `<=` here: the demand is an upper limit, not an obligation. If a delivery would lose money, the model is allowed to deliver less than requested. Naturally, you could again use the keys of the dictionaries, but you could also use the vectors with the names from the DataFrames, e.g. `requested.solar_farm` and `available.supplier`, as both contain exactly the same names. You could even work with ranges from the beginning, e.g. `1:length(available.supplier)` - but working with names is often more convenient.

```
# YOUR CODE BELOW
# Hint: For each solar farm j, the sum of X[i,j] over all warehouses i
# has to be lower than or equal to requested_dict[j]
```

```
# Test your answer
@assert all(
    is_valid(transport_model, restrictDemand[j]) for j in requested.solar_farm
```

```
) "The model should contain one constraint restrictDemand for each solar farm."
@assert normalized_rhs(restrictDemand["b_1"]) == requested_dict["b_1"] "The
right-hand side of the constraint for solar farm b_1 should be its requested
truckloads, $(requested_dict["b_1"])."
println("Demand constraint defined successfully!")
```

And that's it! We have now defined the model and can start optimizing.

Section 5 - Solving the model

Exercise 5.1 - Start optimization

Start the optimization as usual by calling the `optimize!` function on the model instance.

```
# YOUR CODE BELOW


# Test your answer
@assert termination_status(transport_model) == MOI.OPTIMAL "The termination
status should be OPTIMAL but is $(termination_status(transport_model))"
@assert isapprox(objective_value(transport_model), 100373406; atol = 1e-4) "The
optimal profit should be 100373406 but is $(objective_value(transport_model)).
Check the objective and the constraints of your model."
println("Model optimized successfully! The maximal profit is ",
objective_value(transport_model), ".")
```

Now, we can access the values of the variables at the optimal solution. But remember, we defined the variables with the keys of the dictionaries, so we need to convert the result back to a DataFrame. If we just look at parts of the variable container, we only see the variables themselves, not their optimal values.

```
first(X,5)
```

Thus, we need to use the `value` function to extract the values from the variable.

```
transport_values = value.(X)
```

The result is a `DenseAxisArray{Float64,2,...}` with index sets. To convert it to a DataFrame, we just need to iterate over the keys of the dictionaries and store the values in a new DataFrame. As we are not interested in values which are zero, we can skip those.

First, we need to initialize an empty DataFrame with the correct column names and column types.

```
transport_df = DataFrame(
    supplier = String[],
    solar_farm = String[],
```

```
    truckloads = Float64[]
)
```

Then, we can iterate over the keys of the dictionaries and store the values in the DataFrame if they are greater than zero.

```
for i in keys(available_dict)
    for j in keys(requested_dict)
        if transport_values[i,j] > 0
            push!(transport_df, (
                supplier = i,
                solar_farm = j,
                truckloads = transport_values[i,j]
                )
            )
        end
    end
end
```

Finally, we can print the first few rows of the transportation plan to check if it looks correct.

```
println("Beginning of the transportation plan:")
first(transport_df,5)
```

> **Note**
> Although the above code looks rather complicated, it is essentially just iterating over the keys of the dictionaries and storing the values in a new DataFrame. This is a common pattern in optimization, as we often want to convert the result of an optimization problem into a more convenient format for reporting or further processing.

> **Tip**
> We defined $X_{i,j}$ as a continuous variable, so fractional truckloads would be allowed in the plan. If you look at the results, you will see whole truckloads everywhere anyway - a well-known property of transportation problems with whole-numbered supply and demand. In other problems, you might have to decide whether fractional values are acceptable or whether you need integer variables.

Section 6 - Interpreting the solution

Remember the margin of 4700 per truckload from Section 1? A route is only worth using if its transportation costs are below this margin - otherwise, every truckload on that route loses money. Let's check how many routes that rules out:

```
margin = revenue - varCosts
unprofitable_routes = count(travelCosts.costs .> margin)
println("Unprofitable routes: ", unprofitable_routes, " of ",
nrow(travelCosts))
```

```
Unprofitable routes: 66293 of 100000
```

Two thirds of all routes are unprofitable! The optimal plan simply avoids them: if you check `nrow(transport_df)`, you will see that only 1,075 of the 100,000 possible routes are used at all. This is exactly why our demand constraint uses $\leq$ instead of $=$. With $=$, the model would be forced to serve every solar farm from whatever routes are needed - even at a loss.

Exercise 6.1 - Check the delivered truckloads

Does that mean some solar farms receive less than they requested? Find out yourself: compute the total number of truckloads delivered in the optimal plan and save it in a variable called `total_delivered`.

```
# YOUR CODE BELOW
# Hint: Use sum() on the truckloads column of transport_df
```

```
# Test your answer
@assert isapprox(total_delivered, sum(requested.truckload_demand); atol = 1e-4)
"total_delivered should be $(sum(requested.truckload_demand)) but is
$(total_delivered). Did you sum the truckloads column of transport_df?"
println("Correct! All ", sum(requested.truckload_demand), " requested
truckloads are delivered.")
```

Surprised? Although two thirds of all routes are unprofitable, every solar farm still receives everything it requested. Two properties of our data explain this: every solar farm has at least one route with costs below the margin (the cheapest route to any farm costs at most 1,932), and the total supply of 30,321 truckloads exceeds the total demand of 28,014. Keep in mind that this is a property of this particular data set, not a guarantee of the model. If the margin were smaller - say, the variable costs rose above 10,000 - some farms would no longer have any profitable route, and the model would leave their demand unmet.

Conclusion

In this tutorial, we have modelled and solved a transportation problem with real data using JuMP. We have learned how to use dictionaries to store and access the data, which will be useful for more complex models. We have also seen why the model maximizes

profit with demand as an upper limit instead of forcing every delivery: most routes are simply not worth using. If you have any questions, feel free to ask me via email!

Solutions

You will find the solutions to all exercises online here in the `solutions` folders for each part. However, I strongly encourage you to work on these exercises independently without searching explicitly for the exact answers to the exercises. Understanding someone else's solution is very different from developing your own. Use the lecture notes and try to solve the exercises on your own. This approach will significantly enhance your learning and problem-solving skills.

Remember, the goal is not just to complete the exercises, but to understand the concepts and improve your programming abilities. If you encounter difficulties, review the lecture materials, experiment with different approaches, and don't hesitate to ask for clarification during class discussions.